

(PROGRESS REPORT)

Học phần: Operating Systems [Multimedia] - 20252

Mã số dự án (Project No): 25236

Tên dự án (Project title): Optimize pipe implementation to improve IPC throughput

Thiết kế (Design thinking):

Quản lý tác vụ (Task management): [Link JIRA](#)

Thành viên (Members): Nguyễn Sỹ Tuấn, Lê Huy Sơn, Vũ Tiến Long

Giai đoạn báo cáo (Working period): Từ: 5/4/2026 Đến: 12/4/2026

1 Summary of latest progress

(Viết khoảng 200-300 từ để tóm tắt quá trình thực hiện trong giai đoạn này và tổng mức độ hoàn thành hiện tại)

Sau khi đã nộp xong bản báo cáo tuần trước, chúng tôi đã thống nhất lại đơn vị của IPC throughput sẽ là MB/s, và hệ điều hành chúng tôi sử dụng sẽ là xv6. **Lý do:** trong các task chúng tôi phải làm bao gồm: Circular buffer, Increase Size, Optimize Sync thì chúng tôi tìm hiểu thì IPC trong Circular Buffer hoạt động theo cơ chế Shared-Memory (đơn vị MB/s) còn trong Synchronization thì hoạt động theo cơ chế Message Passing (đơn vị messages/s). Tuy nhiên pipe lại truyền theo byte (byte stream) nên đơn vị chính mà chúng tôi sẽ sử dụng là Byte/s (hoặc MB/s, GB/s,...). (Note: $1\text{MB/s} = 10^6\text{B/s}$ còn $1\text{MiB/s} = 2^{20}\text{B/s}$, có sự khác nhau về cơ số).

Để có thêm sự hiểu biết về project này, chúng tôi quyết định là sẽ viết thêm một bản nhật ký, ghi lại các quá trình và kiến thức chúng tôi đã tiếp thu được trong quá trình làm project này. Dưới đây là một số cách truy cập:

Đường link: [Nhật ký kỹ thuật](#)

Drive (sẽ được cập nhật hàng tuần): [Nhật ký kỹ thuật](#)

Chúng tôi cũng đã hoàn thành về việc đã giao ở tuần trước: bao gồm thống nhất lại kiến thức về IPC, IPC mechanism, tìm hiểu cơ bản về các task: circular size, increase size, optimize sync. Cụ thể của các task này sẽ ở phần **Review of technical activities**

Như vậy chúng tôi đã hoàn thành các nhiệm vụ sau:

1 Xác định bài toán

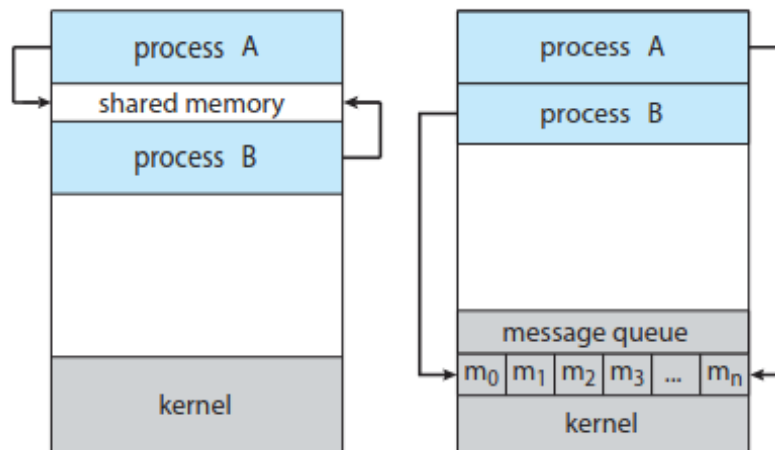
2 Tìm hiểu về IPC

3 Tìm hiểu về các task cơ bản trong bài toán

2 Review of technical activities

Activity 1: IPC

Các tiến trình đang thực thi gồm 2 loại: independent(không chia sẻ dữ liệu với các tiến trình khác) và cooperating(có sự chia sẻ dữ liệu với các tiến trình khác). Trong đó các tiến trình hợp tác (cooperate) cần có cơ chế IPC để trao đổi dữ liệu. Các mô hình IPC: Shared-Memory, Message-passing, Signal. Tuy nhiên thì ở project này thì chúng tôi sẽ tìm hiểu chính về 2 cơ chế là Message-passing và Shared-Memory.



Hình 1: 2 cơ chế IPC

Shared-Memory: Có một vùng bộ nhớ được chia sẻ bởi các tiến trình cooperate, nằm trên không gian địa chỉ của tiến trình tạo ra vùng bộ nhớ chia sẻ. Bằng cách sử dụng phân đoạn bộ nhớ (memory segment) các tiến trình gắn segment này vào không gian địa chỉ (memory-space) của chúng để giao tiếp với nhau.

Message-passing: Cơ chế này rất đặc biệt, nó cho phép các tiến trình giao tiếp và đồng bộ các hoạt động giữa chúng. Các tiến trình giao tiếp cần phải có một liên kết(link) giao tiếp. Link này có 3 cách thiết lập:

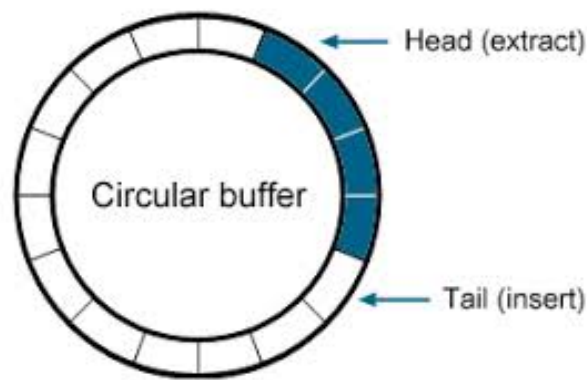
- 1 Direct or indirect thông qua các operations(hoạt động): send(message), receive(message)
- 2 Synchronous or asynchronous communication: thông qua các hoạt động blocking và nonblocking

3 Automatic or explicit buffering: sử dụng các bộ đệm tạm thời để lưu trữ các messages.

Activity 2: Learn about the tasks: Circular buffer. Increase size, optimize synchronization

Tối ưu là một bài toán quan trọng, không sản phẩm nào được tạo ra mà không phải thực hiện bước tối ưu, về trong project về môn hệ điều hành, chúng tôi đã được học qua về user space và kernel space, để đúng nghĩa là một người am hiểu về hệ điều hành, chúng tôi sẽ thực hiện sửa đổi file pipe.c trong kernel(xv6).

Circular buffer: Bộ đệm vòng có cấu trúc như sau:



Hình 2: Bộ đệm vòng

- The Writer advances the HEAD pointer after writing data.
- The Reader advances the TAIL pointer after reading data.
- When either pointer reaches the end, it wraps back to index 0.
- The buffer is FULL when head is one slot behind tail.
- The buffer is EMPTY when head == tail.

Increase Size: Ở task này, size mà chúng tôi cần xác định để tăng lên là bộ đệm và khối dữ liệu.

Đối với bộ đệm: Khi trình ghi lấp đầy bộ đệm, nó sẽ bị chặn cho đến khi trình đọc xử lý xong dữ liệu. Bằng cách mở rộng bộ đệm, trình ghi có thể đẩy nhiều dữ liệu hơn trước khi bị chặn, điều này làm giảm số lần context switch giữa các tiến trình.

Đối với các khối dữ liệu: các khối dữ liệu lớn hơn có nghĩa là ít lệnh gọi hệ thống write()/read() hơn. Mỗi lệnh gọi hệ thống liên quan đến quá trình chuyển đổi

chế độ người dùng -> nhân hệ điều hành -> người dùng.

Synchronization: Đây là bài toán về Data Transfer, chúng tôi đưa ra giải pháp là Zero-copy. Mọi kiến thức về Zero-copy, chúng tôi đã đưa ở mục 2, phần activity 3.

Tuy nhiên, ở trên lớp chúng tôi cũng đã được học về các kỹ thuật synchronization về điều khiển đồng bộ(Concurrency Control) như: Hardware(test-and-set(), compare-and-swap(), atomic variables), Mutex Locks, Semaphores. Các kỹ thuật này tuy làm giảm hiệu năng, nhưng nó làm tăng độ tin cậy(reliability), một khía cạnh cũng khá quan trọng trong việc truyền dữ liệu, chúng tôi chưa biết rằng có nên thêm các kỹ thuật này không.

Activity 3: Một số tài liệu mà chúng tôi đã chép ra về các activities:

1 : [Circular buffer, Increase Size](#)

2 : [Zero-copy](#)

3 : Hoặc tại link JIRA ở trên nhóm tôi cũng đã gửi.

3 Personal contribution

Member 1 (Nguyen Sy Tuan): Viết báo cáo, tổng nhất kiến thức, synchronization

Member 2 (Le Huy Son): Circular buffer

Member 3 (Vu Tien Long): Increase size

4 Developed code review

4.1 Algorithmic development

(Mô tả các thuật toán đã phát triển)

4.2 Source code enhanced and/or revised

(Các đoạn code đã được cải tiến hoặc chỉnh sửa)

4.3 Testing result

(Kết quả kiểm thử, có thể chèn ảnh minh họa bên dưới)

5 TODO

a) **List of remaining issues:** Việc thiếu sót nhất trước khi tìm hiểu về các task này, chúng tôi nên tìm hiểu về pipes trước.

b) **List of tasks to carry out in the next period:**

1 : Tìm hiểu về pipes

2 : Đào sâu vào file pipe.c trong kernel, phân tích

c) **Milestone:** *(Cần làm gì thêm để đạt được mốc tiến độ tiếp theo?)*